EKS Networking

Deep-Dive: kubelet · aws-node · kube-proxy · Cilium

From Scratch · Beginner-Friendly · Production-Grade Terminology

---

This document explains every networking component in your EKS cluster — what it does, how it works, what Cilium replaces, and how prefix delegation solves IP exhaustion. Everything uses Kubernetes-native terminology so you build the right mental model from day one.

Component	Runs On	Primary Job	Replaced by Cilium?
kubelet	Every Node	Node & Pod lifecycle agent	No
aws-node	Every Node	VPC IP allocation (IPAM)	No (chaining mode)
kube-proxy	Every Node	Service → Pod traffic rules	Yes (eBPF replaces)
CoreDNS	Control Plane	DNS resolution for Services	No
Cilium	Every Node	eBPF dataplane + policy + obs	—

1 · KUBELET — The Node's Local Manager

Think of kubelet as a **factory floor manager** sitting on every single node. The API Server (the brain) sends orders: *'run this Pod on your node'*. kubelet receives those orders and makes them happen — and then keeps watching to make sure they stay happening.

What kubelet does — 5 core responsibilities:

- **Watches the API Server** — continuously polls: "which Pods should run on me?"
- **Creates Pods** — talks to the *Container Runtime Interface (CRI)*, i.e., containerd, to pull images and start containers.
- **Health checking** — runs *readinessProbe*, *livenessProbe*, *startupProbe* and restarts containers that fail.
- **Reports Node status** — sends NodeReady / NotReady, CPU pressure, memory pressure back to the API Server continuously.
- **Volume management** — mounts PersistentVolumes (EBS via CSI), ConfigMaps, Secrets into the container filesystem.

■ kubelet does NOT decide where a Pod runs — that is the Scheduler's job. kubelet only enforces the desired state on its own node.

kubelet fact	Detail
Binary location	/usr/bin/kubelet on each EC2 node
Communicates with	API Server via HTTPS (port 6443)
Replaced by Cilium?	No — node lifecycle is always needed
What breaks without it	Pods never start, health checks never run

2 · aws-node (AWS VPC CNI) — Real VPC IPs for Every Pod

In most Kubernetes platforms (Flannel, Calico overlay), Pods get *fake overlay IPs* — addresses that only exist inside the cluster's virtual tunnel network. AWS takes a completely different philosophy:

AWS VPC CNI Philosophy:

```
"Every Pod is a first-class VPC citizen — it gets a REAL VPC IP address."
```

This means your Pod at **10.0.1.45** is reachable directly from anywhere inside your VPC — other EC2 instances, RDS, Lambda — without any overlay tunnel or NAT. That is the power of aws-node.

How aws-node allocates IPs — step by step:

- **Step 1 — Attach ENIs:** aws-node calls the EC2 API to attach extra *Elastic Network Interfaces (ENIs)* to the node.
- **Step 2 — Allocate IPs:** Each ENI can hold several secondary private IPs. aws-node pre-warms a pool of free IPs on the node.
- **Step 3 — Assign to Pod:** When kubelet asks the CNI plugin for an IP for a new Pod, aws-node picks an IP from the warm pool and assigns it to the Pod's *veth pair*.

- **Step 4 — VPC routing:** Because the IP is a real ENI secondary IP, VPC routing tables already know how to reach it — zero extra configuration needed.

Concept	Overlay (Flannel)	AWS VPC CNI
Pod IP type	Virtual / tunnel	Real VPC private IP
Pod-to-Pod routing	Encapsulated (VXLAN/etc)	Native VPC routing
Visible from VPC?	No (NAT required)	Yes — directly
Security Groups on Pod	No	Yes (SG for Pods feature)
Latency	Higher (encap overhead)	Lower (no encap)

aws-node is deployed as a DaemonSet — one Pod per node. That's why you see exactly 2 aws-node Pods for a 2-node cluster.

3 · ENI Limits & Prefix Delegation — Scaling Pod Density

Here is the problem: every EC2 instance type has a hard limit on how many ENIs it can have and how many IPs per ENI. This directly caps the number of Pods per node.

Example limits for a t3.medium:

Resource	Limit	Impact
Max ENIs	3	Only 3 network interfaces
IPs per ENI	6	Max 6 secondary IPs per ENI
Max pod IPs	(3-1) x 6 = 12	~12 Pods max on this node
With prefix delegation	(3-1) x 16 = 32	~32 Pods — significant gain

What is Prefix Delegation?

Instead of allocating **individual IPs** per Pod, AWS allocates an entire **/28 IPv4 prefix** (16 addresses) to each ENI slot. aws-node then hands out individual IPs from that /28 block to Pods.

Without prefix delegation:

```
ENI slot 1 → 10.0.1.10 (1 IP) ENI slot 2 → 10.0.1.11 (1 IP) ENI slot 3 → 10.0.1.12 (1 IP)
```

With prefix delegation:

```
ENI slot 1 → 10.0.1.0/28 (16 IPs) ENI slot 2 → 10.0.1.16/28 (16 IPs) ENI slot 3 →
10.0.1.32/28 (16 IPs)
```

- **Fewer EC2 API calls** — one API call reserves a /28 instead of 16 individual calls.
- **Higher pod density** — more Pods per node without larger EC2 instances.
- **Faster pod launch** — warm IP pool is replenished in bigger chunks.

Secondary CIDR for Pods — When Do You Need It?

Your VPC has a primary CIDR (e.g., 10.0.0.0/16). If your cluster grows large, Pod IPs compete with other VPC resources for that space. The solution is to attach a **secondary CIDR** (e.g., 100.64.0.0/16) to your VPC and configure aws-node to assign Pod IPs from that secondary range — keeping the primary CIDR free for EC2 instances, RDS, etc.

Scenario	Primary CIDR Only	With Secondary CIDR
Pod IPs	Consume 10.0.x.x space	Use 100.64.x.x (separate pool)
Node IPs	Same 10.0.x.x space	10.0.x.x (uncontested)
VPC IP exhaustion risk	High at scale	Low — isolated pools
Recommended for	Dev/small clusters	Production clusters

4 · kube-proxy — Service Traffic Rules (Being Replaced by Cilium)

kube-proxy does **one specific job**: it makes Kubernetes *Services* work. A Service is a stable virtual IP (ClusterIP) that sits in front of a set of Pods. kube-proxy is the component that makes traffic sent to that virtual IP actually reach a real Pod.

How kube-proxy works (traditional iptables mode):

- **Watches API Server** — listens for Service and Endpoints changes.
- **Writes iptables rules** — for every Service, kube-proxy writes iptables NAT rules on every node that say: 'traffic to ClusterIP:port → DNAT to one of [Pod A, Pod B, Pod C]'.
- **Load balancing** — iptables rules distribute traffic across backend Pods (random selection by default).

Traffic flow example:

```
App sends request to: my-service:8080 (ClusterIP: 172.20.0.10) kube-proxy iptables:
172.20.0.10:8080 → DNAT → 10.0.1.45:8080 (Pod B) Packet arrives at: Pod B's real VPC IP
```

The iptables scaling problem:

At small scale iptables is fine. But as your cluster grows:

- **1000 Services × 10 Pods each = 10,000+ iptables rules per node.**
- Every rule change requires a full iptables reload — O(n) latency hit.
- iptables uses sequential rule matching — CPU cost grows linearly.
- Packet loss and latency spikes appear at thousands of Services.

■ This is exactly why Cilium replaces kube-proxy with eBPF — eBPF uses hash-map lookups (O(1)) instead of iptables chains.

kube-proxy mode	Technology	Scale limit	Lookup cost
iptables (default)	Kernel iptables NAT	~1000-2000 Services	O(n) linear
IPVS (optional)	IP Virtual Server	Better, but still limited	O(1) hash
Cilium eBPF	eBPF maps in kernel	Tens of thousands	O(1) hash map

5 · Cilium — The eBPF-Powered Dataplane

Cilium is not just a CNI plugin — it is a full **networking, security, and observability platform** for Kubernetes, built on top of **eBPF** (extended Berkeley Packet Filter). eBPF lets Cilium attach small programs directly into the Linux kernel's network path — no user-space overhead, no iptables, just kernel-speed packet processing.

What is eBPF? (Kernel-level superpower):

eBPF programs run inside the Linux kernel triggered by network events. They can inspect, modify, redirect, or drop packets with near-zero overhead. Traditional tools like iptables work in user space and require kernel/user boundary crossings — eBPF eliminates that.

```
Traditional path: Packet → iptables (kernel) → netfilter hooks → conntrack → forward
Cilium eBPF path: Packet → eBPF hook at XDP/TC layer → hash map lookup → forward Result:
2-3x lower latency, massive scale improvement
```

What Cilium replaces in your EKS stack:

Component / Feature	Replaced by Cilium?	Notes
kube-proxy (iptables)	■ YES — completely	eBPF Service maps replace all iptables rules
Network Policy (basic)	■ YES — and extends it	L3/L4 and L7 (HTTP/gRPC) policies
Service load balancing	■ YES	eBPF maps, O(1) lookup, DSR support
Observability (flow logs)	■ YES — Hubble	Per-flow visibility, Prometheus metrics
kubelet	■ NO	Node/Pod lifecycle — always needed
API Server / Scheduler	■ NO	Kubernetes control plane — not touched
aws-node VPC CNI IPAM	■ NO (chaining mode)	Still allocates ENI/VPC IPs
CoreDNS	■ NO	DNS resolution stays with CoreDNS

Why keep aws-node when you have Cilium?

Cilium CAN fully replace aws-node too (ENI mode), but for your architecture **chaining mode is the correct choice**:

- **Security Groups for Pods** — requires aws-node to attach ENIs directly. This feature lets you assign AWS SGs to individual Pods (not just nodes).
- **AWS-native integrations** — Load Balancer Controller, VPC Flow Logs, and other AWS services expect ENI-attached IPs.
- **Operational simplicity** — aws-node handles IPAM, Cilium handles policy + dataplane. Clean separation of concerns.

6 · Your Production Architecture — Chaining Mode

In chaining mode, aws-node and Cilium work together. Each has a distinct, non-overlapping responsibility:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Your EKS Node ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ aws-node (VPC
CNI) ■ ■ ■ ■ ■■■ Attaches ENIs to EC2 node ■ ■ ■ ■ ■■■ Allocates /28 prefixes (prefix
deleg.) ■ ■ ■ ■ ■■■ Assigns real VPC IPs to Pod veth pairs ■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ Cilium (eBPF) ■ ■ ■ ■
■■■ Replaces kube-proxy (Service routing) ■ ■ ■ ■ ■■■ Enforces NetworkPolicy (L3/L4/L7)
■ ■ ■ ■ ■■■ Hubble: per-flow observability ■ ■ ■ ■ ■■■ Encrypted Pod-to-Pod
(WireGuard/IPSec) ■ ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

■ *When you install Cilium with --set kubeProxyReplacement=strict, you can safely delete the kube-proxy DaemonSet. Cilium takes over 100% of Service routing.*

Data flow in chaining mode — Pod A → Service → Pod B:

- **Pod A** sends packet to Service ClusterIP (e.g., 172.20.0.10:8080).
- **Cilium eBPF hook** intercepts at the veth interface — looks up the ClusterIP in an eBPF hash map (O(1)) — selects backend Pod B.
- **DNAT** happens in the eBPF program — packet is rewritten to Pod B's real VPC IP.
- **VPC routing** (aws-node's contribution) — packet routes natively to Pod B's ENI IP.
- **Cilium on Pod B's node** — enforces any NetworkPolicy rules before delivering to Pod B.

7 · Full Component Comparison

Component	Layer	Owns	Does NOT Own
kubelet	Node	Pod start/stop, health, volumes	Scheduling, networking
aws-node	Network / IPAM	ENI attach, IP alloc, warm pool	Traffic routing, policy
kube-proxy	Network / Services	iptables DNAT rules	Pod IPs, DNS
Cilium	Network / eBPF	Service routing, policy, observability	IP allocation (chaining)
CoreDNS	DNS	Service name → ClusterIP resolution	Routing, policy
Scheduler	Control Plane	Pod → Node assignment	Running pods, networking
API Server	Control Plane	Cluster state, RBAC, etcd	Any dataplane work

8 · Key Takeaways for Your Architecture

- **kubelet** — never replaced. It is the heartbeat of every node. Without it, no Pod ever starts.
- **aws-node** — not replaced in chaining mode. It gives Pods real VPC IPs which unlock Security Groups for Pods and all AWS-native integrations.
- **kube-proxy** — YES, completely replaced by Cilium eBPF. Delete it after installing Cilium with kubeProxyReplacement=strict.
- **CoreDNS** — not replaced. Cilium does not do DNS (yet at scale). CoreDNS resolves my-service.default.svc.cluster.local → ClusterIP.
- **Cilium** — the eBPF dataplane layer that sits above IPAM and below the application. It owns: Service routing, NetworkPolicy (L3/L4/L7), Hubble observability, transparent encryption (WireGuard), and BGP if needed.
- **Prefix Delegation** — enable it before scaling. It multiplies pod density per node by allocating /28 blocks per ENI slot instead of single IPs.
- **Secondary CIDR** — use in production to separate Pod IP space from your primary VPC CIDR. Avoids IP exhaustion as the cluster grows.

Next Practical Step

Run this command to inspect your current VPC CNI config and see if prefix delegation and custom networking flags are set:

```
kubectl describe daemonset aws-node -n kube-system | grep -A2 -E
'ENABLE_PREFIX|AWS_VPC_K8S'
```

Then install Cilium in chaining mode with kube-proxy replacement:

```
helm install cilium cilium/cilium --version 1.15.x \ --namespace kube-system \ --set
eni.enabled=false \ --set kubeProxyReplacement=strict \ --set cni.chainingMode=aws-cni \
--set ipam.mode=cluster-pool
```